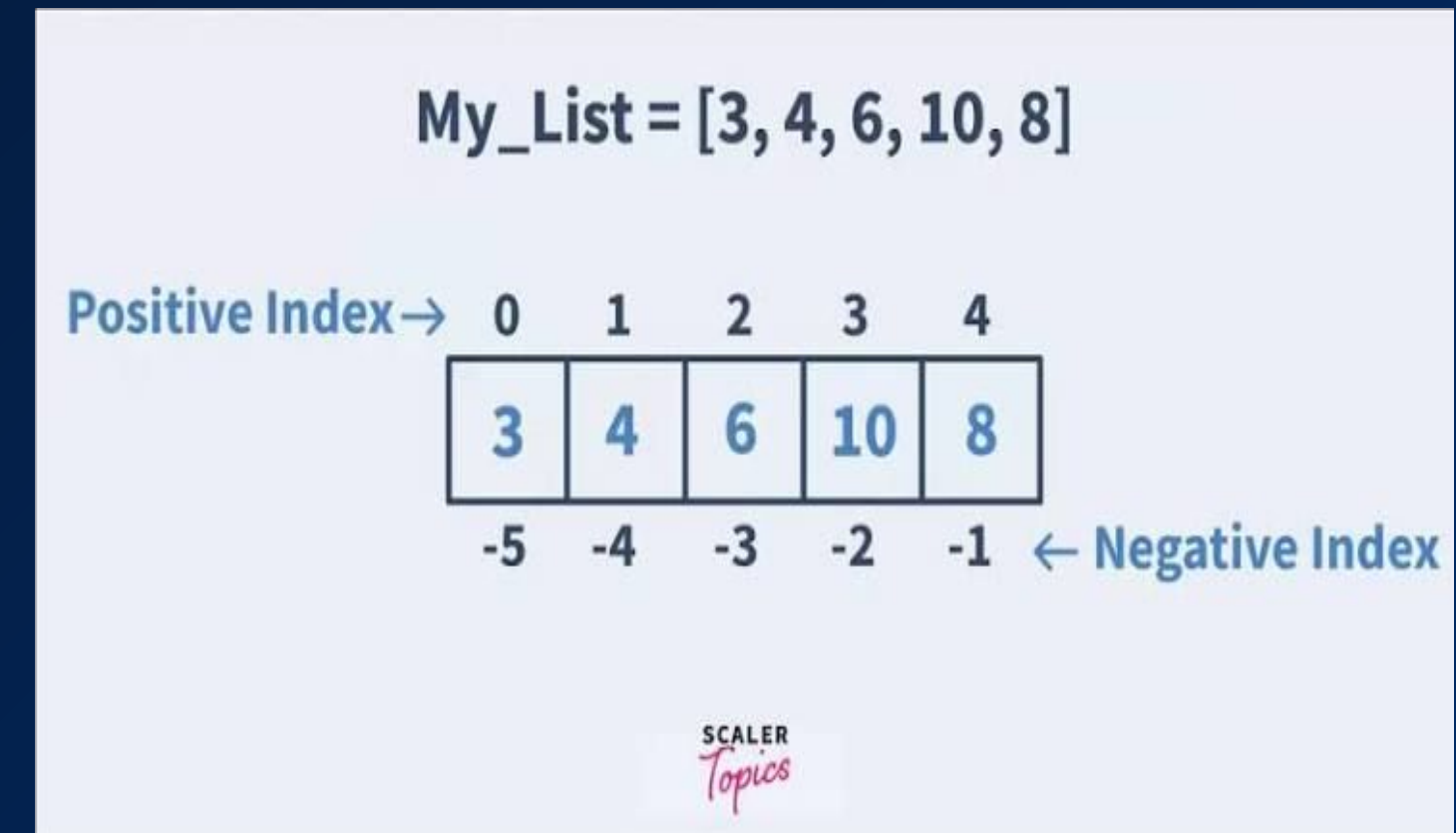


LIST IN PYTHON

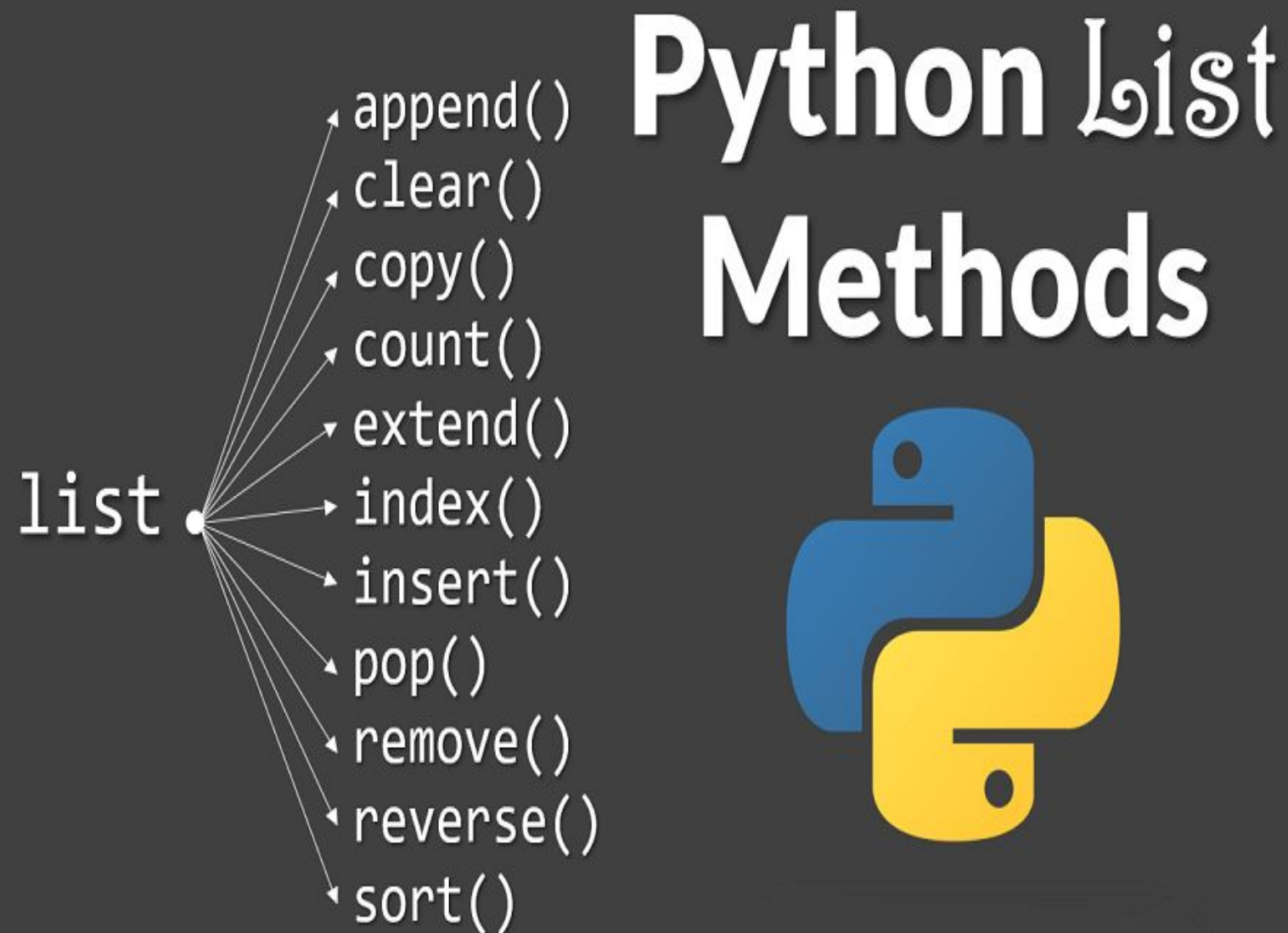
- A List is a collection of multiple items stored in a single variable.
- Lists are ordered, changeable (mutable), and allow duplicate values.
- Lists can store different data types such as integers, strings, and floats.
- Lists are created using square brackets [].

Example:

```
fruits = ["Apple", "Banana", "Mango"]  
print(fruits)
```



Common List Operations



- Accessing Elements: Elements can be accessed using index numbers, starting from 0.
- Adding Elements: `append()`, `insert()`, and `extend()` are used to add single or multiple elements.
- Removing Elements: `remove()`, `pop()`, and `clear()` are used to delete elements from a list.
- Updating Elements: Existing elements can be modified by assigning a new value to their index.
- Finding Length: The `len()` function is used to find the total number of elements in a list.

List Features & Applications

- **Ordered:** Elements are stored in a specific sequence and maintain their order.
- **Mutable:** List elements can be added, removed, or modified after creation.
- **Allows Duplicates:** The same value can appear multiple times in a list.
- **Multiple Data Types:** A single list can contain integers, strings, floats, and other data types.
- **Indexing & Slicing:** Elements can be accessed individually or as a range using indexes.

List Features & Applications

- **Storing Student Marks:** Used to store and manage marks of multiple students.
- **Managing Names & Records:** Useful for storing names, employee details, and other records.
- **Data Collection:** Used to store and organize collections of related information.
- **Data Science & Machine Learning:** Helps store and process datasets and input values.
- **Data Processing:** Commonly used with loops for searching, sorting, filtering, and processing data.

Meet the **TUPLE**

An ordered, immutable collection that groups related values.

💡 CORE IDEA

One variable. Multiple related values.

Groups heterogeneous elements into a single unit.

01 ORDERED

Elements maintain their order and can be accessed by index.

02 IMMUTABLE

Tuple elements cannot be added, removed, or reassigned.

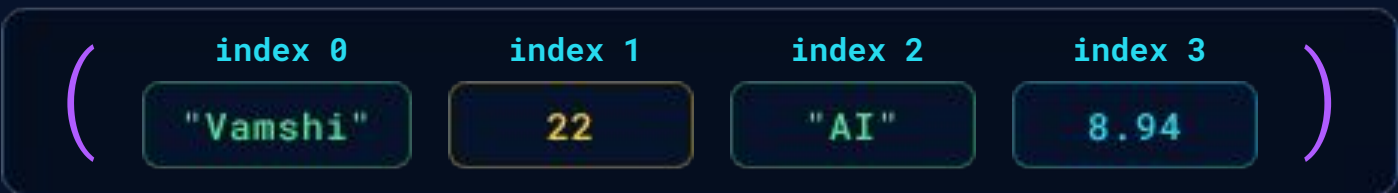
03 HETEROGENEOUS

Different data types can coexist seamlessly in one tuple.

🔗 Fixed sequence model in Python memory

📦 TUPLE ANATOMY & INDEXING

```
student = ("Vamshi", 22, "AI", 8.94)
```



Zero-indexed from left to right. Each position holds a fixed reference.

📌 HOW A TUPLE IS CREATED

</> 1. BASIC SYNTAX

```
t = ("A", "B", "C")
```

Created with parentheses enclosing comma-separated values.

⚠️ 2. SINGLE ITEM

```
t = ( 5 ) ✓ Tuple x = ( 5 ) ✗
                        Integer
```

The comma creates the one-item tuple.

🔧 3. CONSTRUCTOR

```
tuple([1, 2, 3]) → (1, 2, 3)
```

`tuple()` constructor builds tuple from any iterable.

Working with TUPLES

Access, inspect and combine values without changing the original tuple.

01 INDEXING

```
t = ("A", "B", "C", "D") t[0] # → "A" (First element) t[-1] # → "D" (Last element)
```

0	1	2	3
A	B	C	D
-4	-3	-2	-1

Positive indexing starts at 0. Negative indexing starts at -1.

02 SLICING

```
t = (10, 20, 30, 40, 50) t[1:4] # → (20, 30, 40) t[::-1] # → (50, 40, 30, 20, 10) (Reversed)
```

10

 |

20

30

40

 |

50

 → (20, 30, 40)

start : stop : step syntax returns a new tuple without modifying original.

03 OPERATORS

CONCATENATION

$(1, 2) + (3, 4) \rightarrow (1, 2, 3, 4)$

REPETITION

$(1, 2) * 2 \rightarrow (1, 2, 1, 2)$

MEMBERSHIP

$2 \text{ in } (1, 2, 3) \rightarrow \text{True}$

Operators build fresh sequence objects; operands remain untouched.

04 ITERATION

```
for item in t: print(item)
```

A

 →

B

 →

C

 →

D

Tuples are iterable and can be traversed using a for-loop.

✔ ACCESS t[i] ✔

✔ SLICE t[a:b] ✔

✔ SEARCH item in t ✔

✖ MODIFY t[i] = x ✖ (PROHIBITED)

Beyond the BASICS



TUPLE METHODS

Only 2 Methods

```
t.count(x) # → Number of occurrences of value x    t.index(x) # → First matching index of value x
```

These are the only two built-in methods on Tuple objects.



BUILT-IN FUNCTIONS

Python Built-ins

```
len(t) # Length    min(t) # Minimum    max(t) # Maximum    sum(t) # Total sum
```

These are Python built-in functions, NOT Tuple methods.



SWAPPING

```
a, b = 10, 20; a, b = b, a
```

a: 10 → 20 | b: 20 → 10



MUTABILITY NUANCE

Advanced

```
t = ([1, 2], 3); t[0].append(4) # Result: ([1, 2, 4], 3) ✓ Allowed
```

Immutable tuple ≠ immutable objects inside it.



NESTED TUPLES

```
data = (("A", 90), ("B", 85)) data[0][1] # → 90
```

[0] accesses inner tuple, [1] accesses value.

The KEY DIFFERENCE: List vs Tuple

Both are ordered sequences. Mutability is the fundamental difference in memory and usage.

PROPERTY	LIST []	TUPLE ()
Ordering	Ordered	Ordered
Mutability	Mutable (Can change)	Immutable (Cannot change)
Duplicates	Allowed	Allowed
Indexing & Slicing	Supported (0-indexed)	Supported (0-indexed)
Dictionary Key	Cannot be dict keys	Eligible if all elements hashable
Typical Use	Changing / dynamic collections	Fixed records / protected collections

LIST: [10, 20, 30] → [100, 20, 30]
✓ element can change

TUPLE: (10, 20, 30) → (10, 20, 30)
🔒 element cannot be reassigned

⚡ Rule of Thumb: Use a **LIST** when data changes. Use a **TUPLE** when data should stay fixed.



LIVE MUTATION TEST

LIST: [10, 20, 30]

✓ CAN CHANGE

```
numbers[0] = 100 # Result: [100, 20, 30]
```

TUPLE: (10, 20, 30)

🔒 LOCKED

```
numbers[0] = 100 # ✗ TypeError
```

```
TypeError: 'tuple' object does not support item assignment
```

SETS

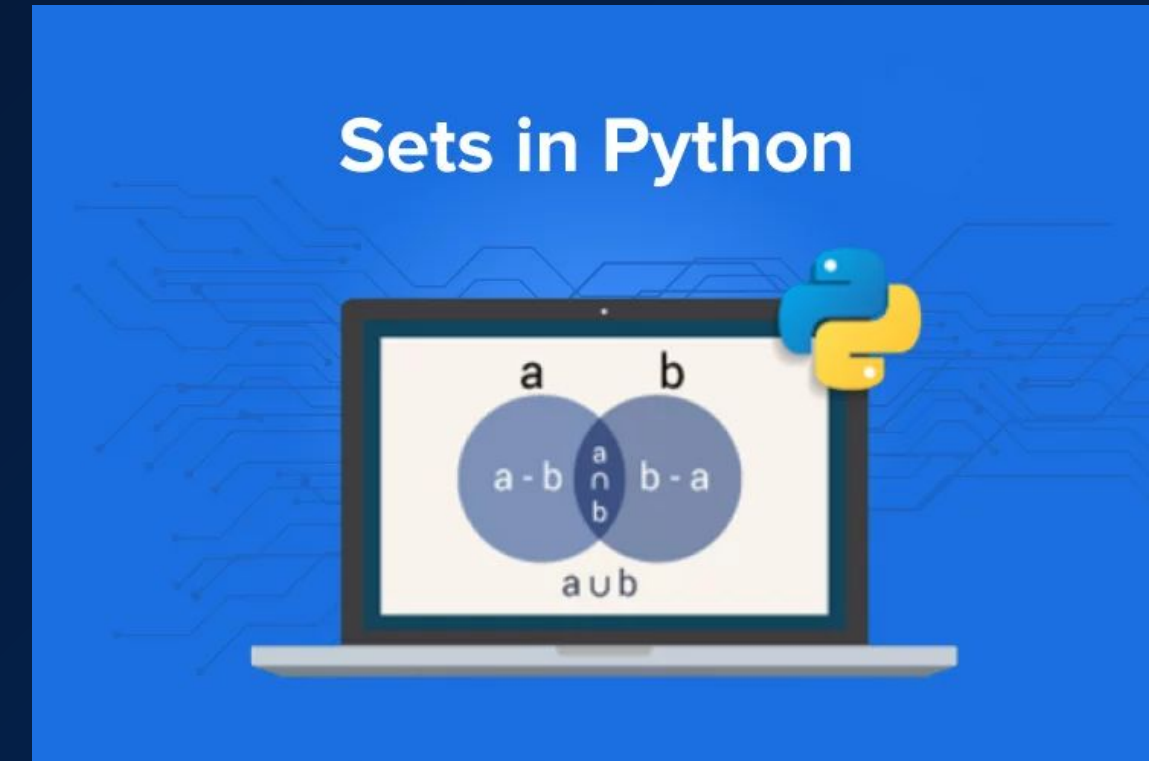
Introduction to Sets

Sets in Python

- ❖ **Unordered** collection of unique elements
- ❖ Created using { }
- ❖ **Duplicates values are removed automatically**
- ❖ **Mutable** — elements can be added or removed
- ❖ Useful for **unique data & mathematical set operations**

```
numbers = {1, 2, 3, 3, 4}  
print(numbers)
```

```
{1, 2, 3, 4}
```



List vs Sets

List

- Ordered
- Support Duplicate
- Indexing(**YES**)

Sets

- Unordered
- No Duplicate
- Indexing(**NO**)

Creating & Modifying Sets

Creating a Set:

```
fruits = {"apple", "banana", "orange"}
```

Adding an Element:

```
fruits.add("mango")
```

Adding Multiple Elements:

```
fruits.update(["grapes", "kiwi"])
```

Removing an Element:

```
fruits.remove("banana")
```

Important points:

- ★ Sets do not allow **duplicate** values.
- ★ Sets are **unordered**, so indexing like `set[0]` is not supported.
- ★ **discard()** can also remove an element without raising an error if it doesn't exist.

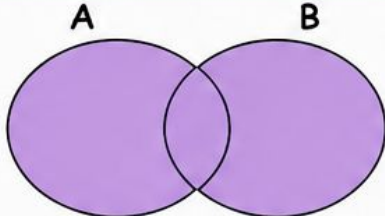
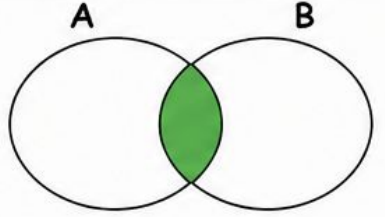
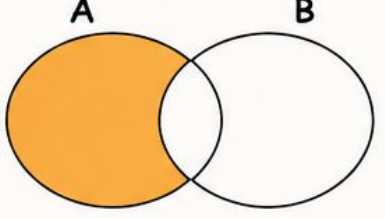
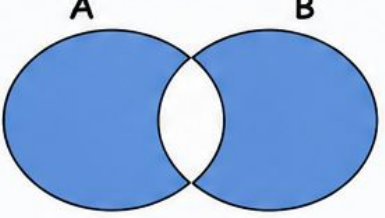
```
numbers = {1, 2, 3, 4}  
numbers.discard(3)  
print(numbers)
```

```
{1, 2, 4}
```

Set Operations

Set Operations in Python

$A = \{1, 2, 3, 4\}$ $B = \{3, 4, 5, 6\}$

Operation	Python	Result	Venn Diagram
Union (All elements from both sets)	$A \mid B$	$\{1, 2, 3, 4, 5, 6\}$	 $\{1, 2, 3, 4, 5, 6\}$
Intersection (Common elements in both sets)	$A \& B$	$\{3, 4\}$	 $\{3, 4\}$
Difference (Elements in A but not in B)	$A - B$	$\{1, 2\}$	 $\{1, 2\}$
Symmetric Difference (Elements in A or B but not in both)	$A \wedge B$	$\{1, 2, 5, 6\}$	 $\{1, 2, 5, 6\}$

Advantages of Sets

- Removes duplicates automatically.
- Fast membership checking using `in`.

Disadvantages of Sets

- No indexing or ordering like lists.
- Can only store hashable (immutable) elements such as numbers, strings, and tuples.

DICTIONARIES IN PYTHON

- > A **dictionary** is a built-in data type in python.
- > It stores key value pairs.
- > Dictionaries are written using {}.
- > Each key should be unique.
- > Dictionaries are mutable, so their data can be changed.
- > They are useful when data needs to be stored with a meaningful identifier

```
a = {"name": "Rahul"}  
print(a["name"])
```


DICTIONARIES OPERATIONS

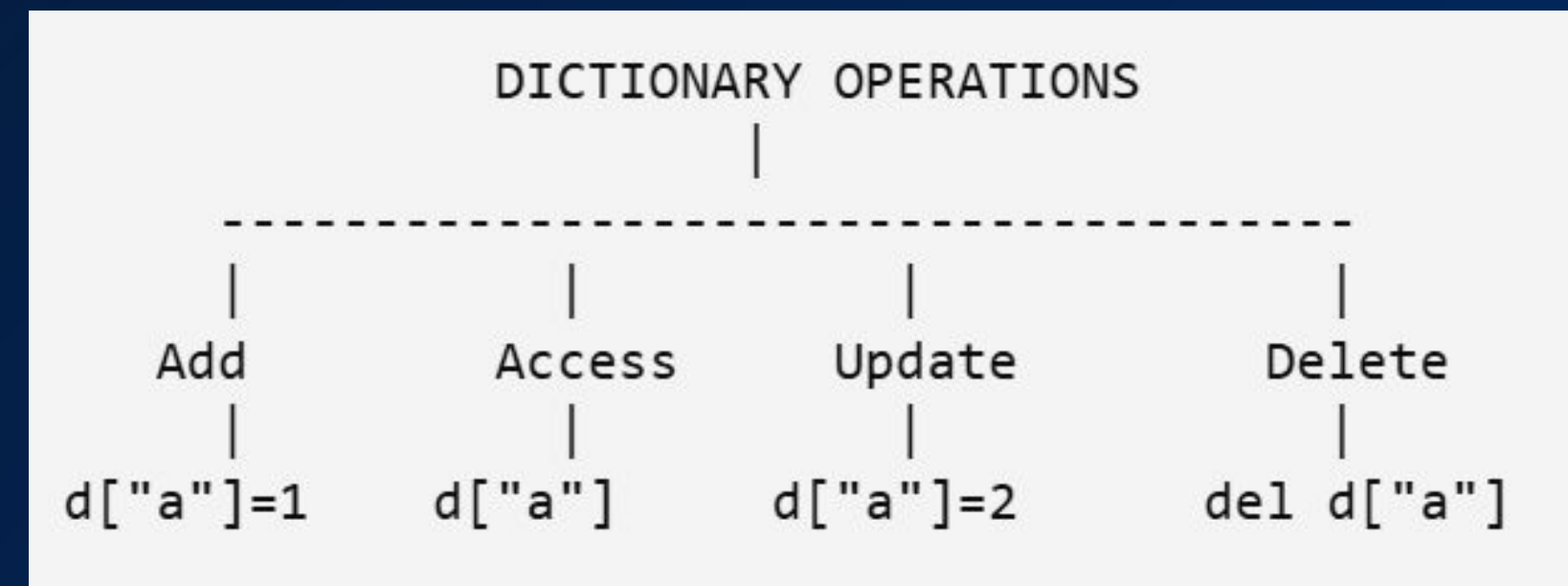
-> **Accessing** : values can be retrieved using their keys.

-> **Adding** : New key-value pairs can be added.

-> **Updating** : Existing values can be changed.

-> **Removing** : Key-value pairs can be deleted.

-> **Checking** : We can check whether a particular key exists in a dictionary



Methods & Applications

- > `keys()` - returns all the keys.
- > `values()` - returns all the values.
- > `items()` - returns all the key-value pairs.
- > `get()` - safely retrieves a value.
- > `update()` - adds or modifies key-value pairs.
- > `pop()` - removes a specified key-values pair

- > Provides fast access to data using keys
- > Makes data easy to organize and understand.
- > Allows data to be modified easily.
- > It can store different types of values.
- > Useful for representing structured information.

List & Tuple – Methods and Operations

List Methods

Method	Syntax	Purpose
<code>append()</code>	<code>list.append(item)</code>	Adds item at end
<code>insert()</code>	<code>list.insert(index, item)</code>	Adds item at position
<code>remove()</code>	<code>list.remove(item)</code>	Removes an item
<code>pop()</code>	<code>list.pop(index)</code>	Removes item by index
<code>sort()</code>	<code>list.sort()</code>	Sorts the list
<code>reverse()</code>	<code>list.reverse()</code>	Reverses the list
<code>count()</code>	<code>list.count(item)</code>	Counts occurrences
<code>index()</code>	<code>list.index(item)</code>	Finds item position

List Operations

Concatenation

`list1 + list2`

Repetition

`list1 * 2`

Membership

`item in list1`

Indexing

`list1[index]`

Slicing

`list1[start:end]`

Tuple Methods & Operations

Tuples are **immutable** — they only support `count()` and `index()`. Operations mirror lists: `+`, `*`, `in`, indexing, and slicing.

Set – Methods and Operations

Set Methods

Method	Syntax	Purpose
<code>add()</code>	<code>set.add(item)</code>	Adds an item
<code>update()</code>	<code>set.update(iterable)</code>	Adds multiple items
<code>remove()</code>	<code>set.remove(item)</code>	Removes item (error if missing)
<code>discard()</code>	<code>set.discard(item)</code>	Removes item (no error)
<code>pop()</code>	<code>set.pop()</code>	Removes arbitrary item
<code>clear()</code>	<code>set.clear()</code>	Removes all items

Set Operations

Union $A \cup B$

Combines both sets

Intersection $A \cap B$

Common elements only

Difference $A - B$

Elements only in A

Symmetric Diff. $A \oplus B$

In either set, not both

Example:

$A = \{1, 2, 3\}$

$B = \{3, 4, 5\}$

$A \cup B = \{1, 2, 3, 4, 5\}$

$A \cap B = \{3\}$

$A - B = \{1, 2\}$

$A \oplus B = \{1, 2, 4, 5\}$

Dictionary – Methods and Operations

Dictionary Methods

Method	Syntax	Purpose
<code>keys()</code>	<code>dict.keys()</code>	Returns all keys
<code>values()</code>	<code>dict.values()</code>	Returns all values
<code>items()</code>	<code>dict.items()</code>	Returns key-value pairs
<code>get()</code>	<code>dict.get(key)</code>	Gets value of a key
<code>update()</code>	<code>dict.update(data)</code>	Adds or updates items
<code>pop()</code>	<code>dict.pop(key)</code>	Removes a key-value pair
<code>popitem()</code>	<code>dict.popitem()</code>	Removes last pair
<code>clear()</code>	<code>dict.clear()</code>	Removes all items

Dictionary Operations

Access

`dict[key]`

Add / Update

`dict[key] = value`

Delete

`del dict[key]`

Check Key

`key in dict`

Length

`len(dict)`



Example:

```
student = {"name": "Rahul", "age": 20}
student["name"] # Access → "Rahul"
student["age"] = 21 # Update
student["city"] = "Mumbai" # Add
del student["age"] # Delete
```


Things Inside Things?!

NESTED DATA STRUCTURES

Python Data Structures – Quick Recap

These are the building blocks of data in Python.

Data Structure	Example	Can be Nested?
 List Ordered collection of items	[10, 20, 30] ↑ Items in a specific order	 Yes
 Dictionary Key-value pairs	{"name": "Malar"} ↑ Key maps to a value	 Yes
 Tuple Ordered collection of items (immutable)	(10, 20, 30) ↑ Fixed and cannot change	 Yes
 Set Unordered collection of unique items	{10, 20, 30} ↑ No duplicates, no order	 Yes

Imagine a student record containing:

- Personal details
- Courses
- Scores

Instead of keeping everything separately:

LIST

↓

DICTIONARY — Student

↓

DICTIONARY — Courses

↓

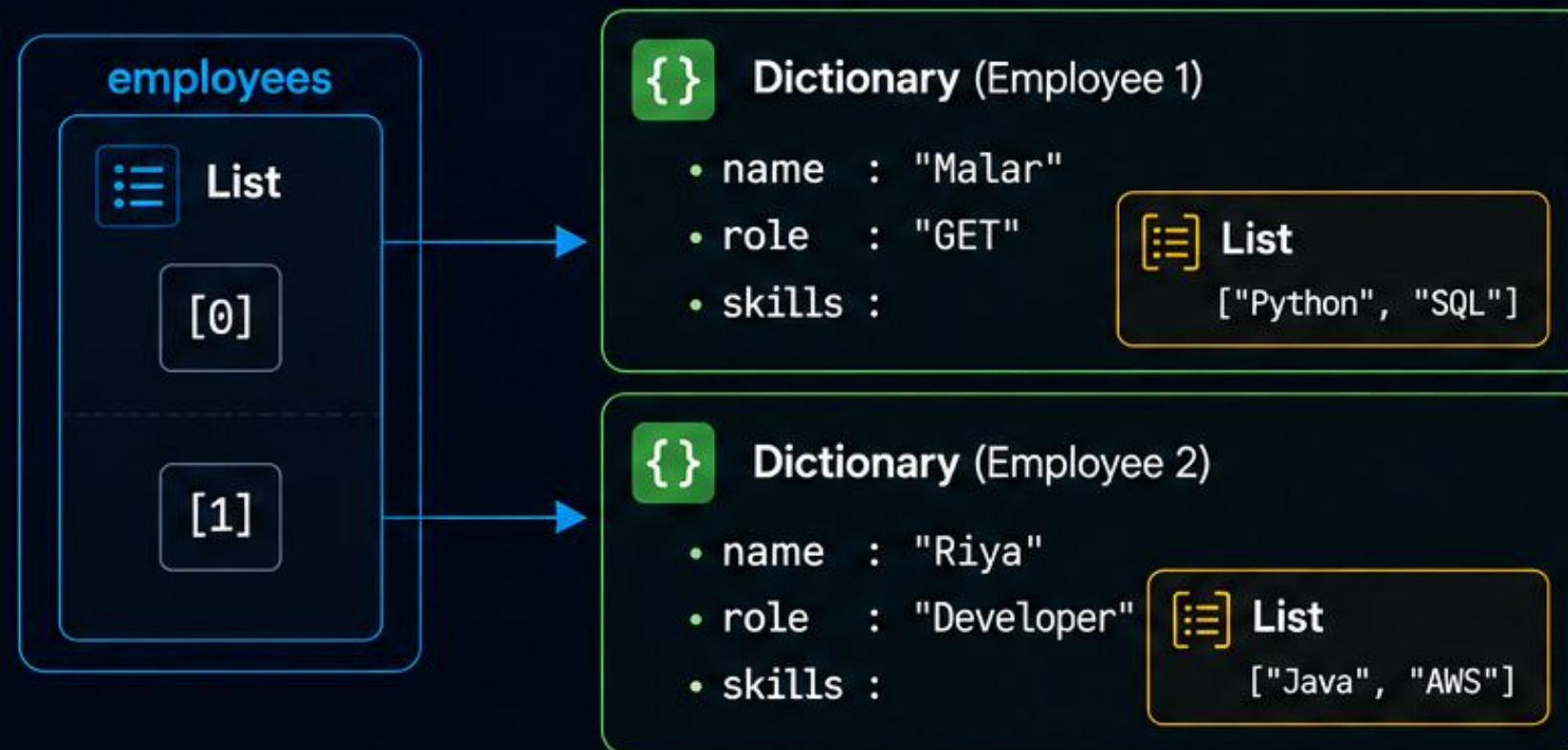
KEY + VALUE — "SQL": 88

When one data structure contains another data structure inside it.

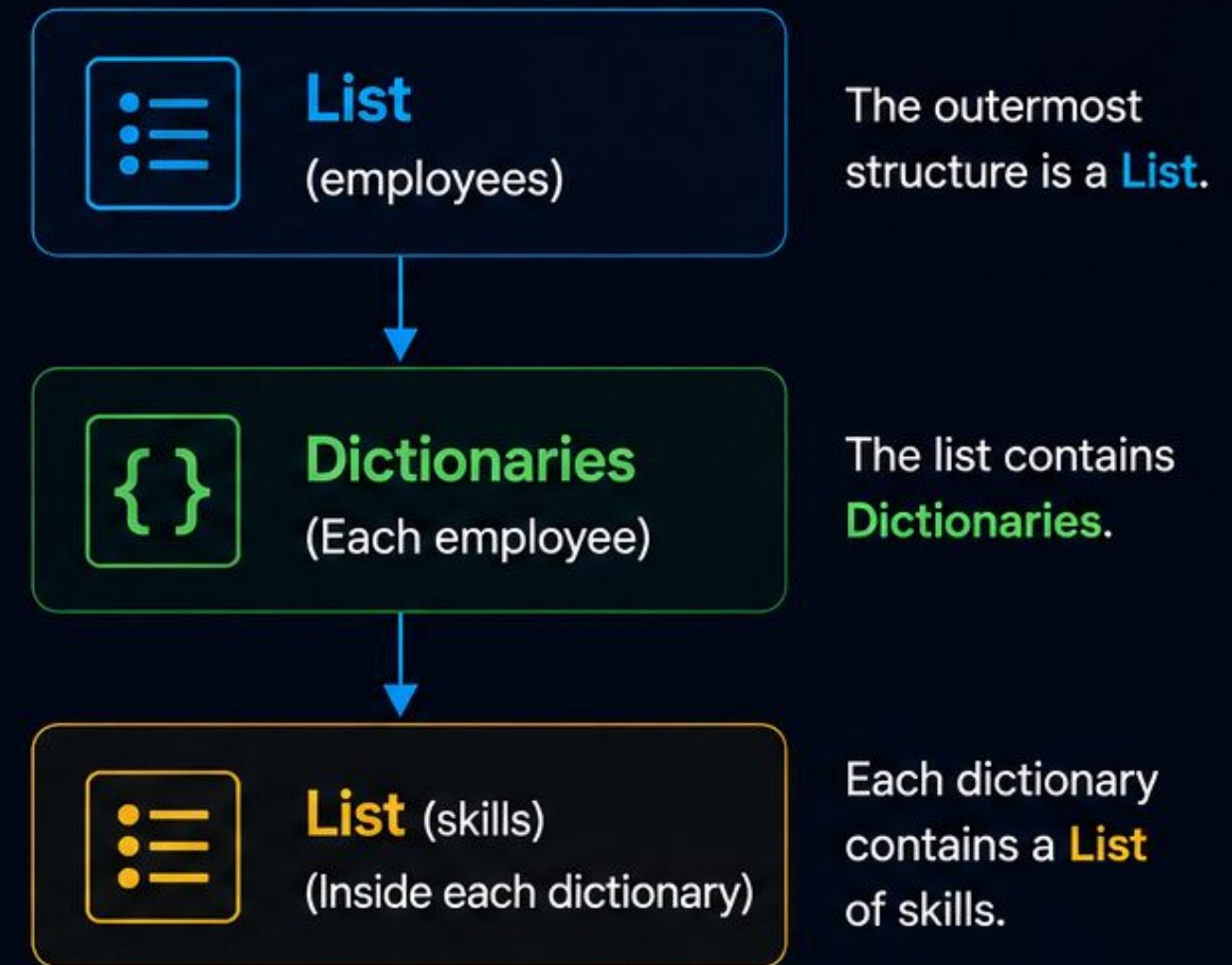
Employees Data (Nested Structure)

```
employees = [  
  {  
    "name": "Malar",  
    "role": "GET",  
    "skills": ["Python", "SQL"]  
  },  
  {  
    "name": "Riya",  
    "role": "Developer",  
    "skills": ["Java", "AWS"]  
  }  
]
```

Visual Representation



What are we **actually** looking at?



In short:

List → contains **Dictionaries**
Each **Dictionary** → contains a **List**

Nested Challenge: Can You Trace the Data?

```
students = [  
    {  
        "name": "Arjun",  
        "courses": [  
            {"name": "Python", "score": 92},  
            {"name": "SQL", "score": 88}  
        ]  
    },  
    {  
        "name": "Riya",  
        "courses": [  
            {"name": "Python", "score": 85},  
            {"name": "SQL", "score": 91}  
        ]  
    }  
]
```

1. What will this return?

```
students[1]["courses"][0]["score"]
```

- A) 88
- B) 85
- C) 91
- D) Python

2. How would you access Riya's SQL score?



ITERATION AND MANIPULATION

Iteration means repeating a set of instructions, usually using loops and the loop changes checking if the condition is being given or not.

Python mainly has two types of loops:

- for loop → repeat for each item in a sequence
- while loop → repeat while a condition is true

```
</> Python
fruits = ["apple", "banana", "mango"]

for fruit in fruits:
    print(fruit)
```

Result:

```
apple
banana
mango
```

```
</> Python

i = 1

while i <= 5:
    print(i)
    i += 1
```

Output:

```
1
2
3
4
5
```


MANIPULATION

Manipulation means changing, modifying, adding, removing, or rearranging data.

The functions used to manipulate are

- `append()` - This adds a additional value in the list
- `insert()`- This adds a value but at the designated place we allot
- `remove()` - This removes the value
- `pop()` - This helps in the removing the value with the position value

```
Python
cars = ["BMW", "Audi", "Toyota"]

print("Original list:", cars)

cars.append("Mercedes")
print("After append:", cars)

cars.insert(1, "Ferrari")
print("After insert:", cars)

cars.remove("Audi")
print("After remove:", cars)

cars.pop(2)
print("After pop:", cars)
```

Result

```
Original list: ['BMW', 'Audi', 'Toyota']
After append: ['BMW', 'Audi', 'Toyota', 'Mercedes']
After insert: ['BMW', 'Ferrari', 'Audi', 'Toyota', 'Mercedes']
After remove: ['BMW', 'Ferrari', 'Toyota', 'Mercedes']
After pop: ['BMW', 'Ferrari', 'Mercedes']
```

STRING MANIPULATION

The string values can be manipulated

The functions used in the string manipulation are:

- `upper( )` - converts the string to uppercase
- `lower( )` - converts the string to lowercase
- `replace( )` - this is used to replace one word with another
- `split ( )` - this is used to split the string based on the criteria given

```
Python
text = "Python Programming"

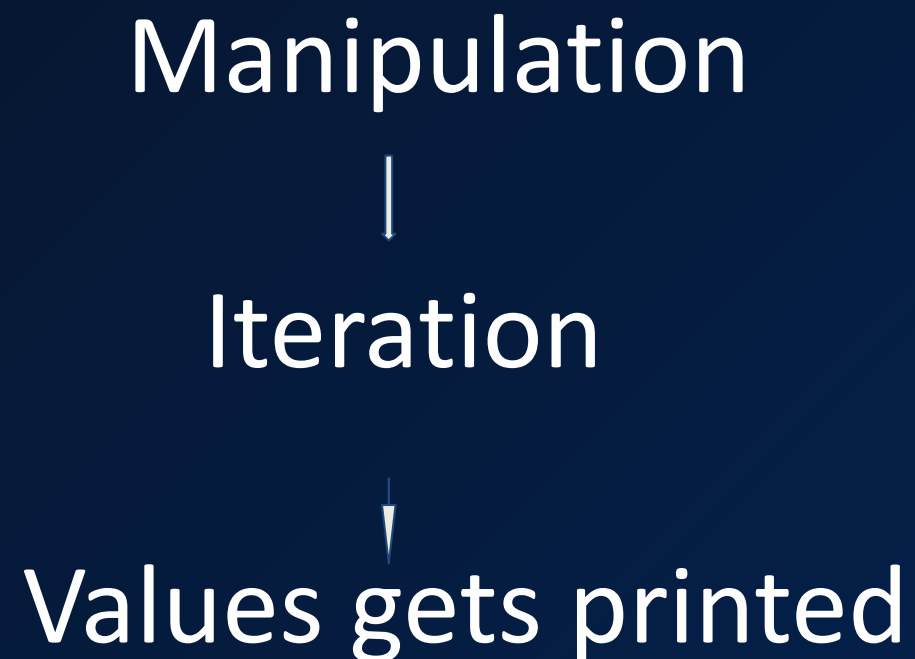
print("Original String :", text)
print("Uppercase       :", text.upper())
print("Lowercase       :", text.lower())
print("Replace         :", text.replace("Python", "Java"))
print("Length          :", len(text))
print("Split           :", text.split())
print("First 6 chars    :", text[:6])
print("Last 11 chars    :", text[7:])
print("Reversed        :", text[::-1])
```

Result:

```
Original String : Python Programming
Uppercase       : PYTHON PROGRAMMING
Lowercase       : python programming
Replace         : Java Programming
Length          : 18
Split           : ['Python', 'Programming']
First 6 chars    : Python
Last 11 chars    : Programming
Reversed        : gnimmargorP nohtyP
```

ITERATION AND MANIPULATION

The manipulation happens in the list and then the iteration happens from the 1st till the end of the list.



```
</> Python Run

numbers = [1, 2, 3, 4, 5]

for i in range(len(numbers)):
    numbers[i] = numbers[i] * 2

print("Original list: [1, 2, 3, 4, 5]")
print("After manipulation:", numbers)
```

Result

```
Original list: [1, 2, 3, 4, 5]
After manipulation: [2, 4, 6, 8, 10]
```